Lazy range-fetch mounting of runtime images

TBK SPEC 39:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 39:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. Why — and why LimniFS fits lazy pull better than eStargz fits tar 4

2. The seam: a mount-source kind, NOT a format backend 5

3. The trust decision (locked): per-block-group digests at read, whole-file sha256 at seal 6

4. Cache design and the store-state amendment 8

5. The seal path and the state machine 9

6. Failure semantics (named, never a partial mount) 10

7. UX: mount mode, progress, doctor 11

8. Interaction with spec 38 (OCI) 13

9. What does NOT change (locked) 13

10. Test plan (the tiers of spec 14; each failure above maps to one tier) 15

11. Open questions (each with this spec's recommendation) 16

Implementation plan (spec PR first; each PR independently mergeable) 16

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

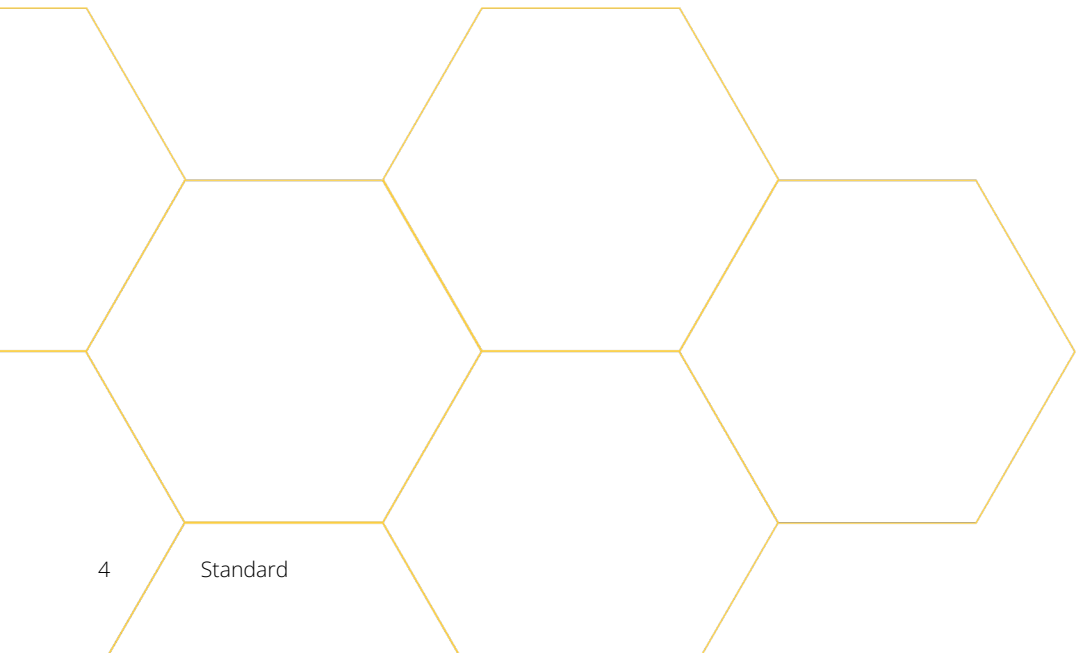
Foreword

Status: SHIPPED (locked direction 2026-09-30, tebako#696 — spec-first per spec 14). PRs 1–6 shipped the spec + schemas, the Range transport (tebako-http), the byte-source seam + caching remote source + lazy store record (tfs/tpkg), the loader + driver wire (resolve plan lazy arm, shim/bootstrap wire, the state-detecting env-image mount, the background seal thread, `tebako cache seal` + the listing/doctor surfaces), the publish path (in-process blksum generation + the additive `image.blksum` index field), and the OCI range arm + the bench arms (§8, spec 27 §10.5). Since then the downstream legs have landed: the lazy-capable runtime releases, the §10 parity data, the default flip (§7 — lazy-on by default, locked), and the cached-blksum arm (§3 — the install-time sidecar prefetch that keeps the mount-open zero-network). Nothing here changes what a shipped loader, driver, or resolver does with a store that carries none of the new artifacts: a sidecar-less release takes the loud eager fallback and every published image remains runnable exactly as today.

Normative specification of lazy mounting: the runtime env image is mounted over HTTPS with HTTP Range reads, the machine cache is seeded block-group by block-group on demand, and a background seal pass turns the partially-seeded entry into an ordinary, fully-verified store entry. This spec owns the remote-locator hook recorded in spec 20 §7 ("Remote locators (http/s3/ipfs) → a future streaming spec").

1. Why — and why LimniFS fits lazy pull better than eStargz fits tar

First run of a lean (shared-runtime) package downloads the whole runtime pair (100–300 MB) before one user byte executes. Containerd's lazy-pull line (eStargz/SOCI/nydus) proved the fix at the format level; Harter et al. (FAST'16) measured pulls at 76% of container start time while only 6.4% of pulled bytes are ever read.



© 2026 The Tebako Project

eStargz needed a **format change** to make lazy pull possible: tar is a streaming format with no index, so stargz added a TOC, per-member gzip streams, and landmark headers — a new wire shape that only new images carry. LimniFS needs none of that. It is seekable **by construction** (spec 20 §4): the inline metadata blob resolves path → inode → drop slices; the slab record tables carry each drop's byte offset and length; `pread` materializes only the drops intersecting the requested window (a seekable-container drop decodes only the covering 256 KiB frames). Every LimniFS image ever published is already a lazily-fetchable set of byte ranges. The lazy feature is therefore a READER-side change only — no format change, no new in-image section, no republication of image bytes (§9).

Scope, locked for v1: the runtime **env image** (`TEBAKO_RUNTIME_IMAGE`) in managed dispatch (tebako-shim) and standalone dispatch (tebako-bootstrap). The interpreter exe is always fetched whole — the kernel execs a file, not a mount. Payload slices ride the same machinery later (§11).

2. The seam: a mount-source kind, NOT a format backend

The issue's sketch ("a new TFS backend `http-range` beside zip/dwarfs-t/squashfs/tar") is refined against the shipped seam. The spec 11 `Backend` trait (`crates/tfs/src/backend.rs`) is **path-addressed and format-keyed**: `stat/pread/read_dir/read_link` by in-image path, selected by magic detection (spec 20 §3). Transport is not a format — `format_id` answers only "how do I read these bytes" (orthogonality law, spec 00). An HTTP Range transport placed at the Backend seam would either fake a format (wrong) or re-implement one (duplicated authority).

The correct layer is the one below the format backend. Spec 11 §5's mount-source kinds (host-file, memory, file-region, VFS-file-region) all serve positioned byte reads into the mount constructors (`crates/tfs/src/mount.rs`). This spec adds a **fifth mount-source kind: remote-range**, behind a byte-source abstraction:

FIGURE 1

```
transforms (COW / ENC — Rust TFS only, spec 00 invariant 5,
spec 11 §4)
  → format backend (limnifs; detection unchanged)
    → ByteSource: host file | memory | file region | VFS
region | REMOTE-RANGE
```

- **The transforms-law answer (precise):** the lazy source sits UNDER the format backend, at the byte layer. COW and ENC stack ABOVE the format backend exactly as today —

© 2026 The Tebako Project

a COW overlay over a lazily-mounted env image writes to its HostDir store and never touches the network; an ENC transform decrypts bytes the source already supplied. The transforms law is untouched: the remote source is read-only, and no backend learns to write.

- **The Backend trait is unchanged.** No new trait methods; detection unchanged (the 512-byte sniff is one small range read at open); `writable()` stays None.
- **The limnifs adapter gains a sourced open** beside `LimnifsBackend::from_image(Vec<u8>)` (`crates/tfs/src/backends_limnifs.rs`): mount-open fetches, through the caching byte source, the manifest prefix (header → feature flags → metadata reference + inline metadata blob → slab index → history — the self-delimiting walk the adapter already performs) and each slab's **record table**; slab drop payload bytes are fetched on demand at `pread` time. Prerequisite (upstream, limnifs repo): `limnifs-core` gains a paged `SlabSource` variant (today: `Memory`/`Mapped` only) and a record-table parse over a positioned reader. Reader-side only — no format change. The dwarfs backend's in-place positioned reads make a dwarfs-over-range source possible later across the C++ callback boundary; recorded, NOT committed (§11).
- **Feature gating (spec 20 §5's rule applied):** the remote-range source is a cargo feature `backend-remote` on `tfs`, default ON for tebako-driver and the toolchain, never in tebako-bootstrap (which never mounts, spec 20 §1 — §9). A lazy mount attempted on a build without the feature fails with the NAMED `ENOTSUP` — the compiled-out rule, unchanged.

3. The trust decision (locked): per-block-group digests at read, whole-file sha256 at seal

The issue's open question — whole-file sha256 at seal vs per-block-group digests at read — is decided: **per-block-group digests at read is the PRIMARY design; the whole-file sha256 verifies at seal as the mandatory second anchor.** The rejected alternative (whole-file verify only at seal) is rejected on the threat model, not on convenience.

Threat model. Today's law (spec 05 §4): verification at fetch/install, never per run — the payload never executes a byte that was not verified against the `.sha256` trust anchor first. A seal-only lazy design breaks that law for the entire seeding window: a malicious or compromised registry (or a TLS-intercepting path the enterprise roots admit, spec 04 §4) could serve self-consistent bad blocks, and the payload would EXECUTE them on first run; the seal would fail loudly only afterwards. That is a strict weakening of the shipped trust posture and is unacceptable. (The image's own content addressing — LimniFS drop ids are `BLAKE3(plaintext)`, spec 20 §1 — detects corruption and bit-rot but is NOT a trust root: a hostile registry serves a self-consistent evil image whose ids match its own evil metadata.)

© 2026 The Tebako Project

The design.

- **Fetch unit = the block group**, a locked **4 MiB** span of the IMAGE bytes (the wire bytes — format-agnostic, so the same source later serves dwarfs; the final group is short). Groups align all cache, verification, and progress accounting (§4, §7).
- **The digest sidecar** — `<image>.blksum.json`, published beside the image by the PUBLISHER (the runtime factory / the feedstock publish leg; `tebako publish` generates it in-process in the same invocation that stages the release bytes): a versioned JSON document (`schema/tpkg-blksum-v1.schema.json`) carrying `schema_version`, `group_size`, `size_bytes`, the whole-image `sha256`, and one sha256 per group. A resolver NEVER derives it (derivation would require the whole file — pointless — and would arrive unanchored). Three callers reach the ONE emitter (`tpkg::lazy::write_sidecar`): `tebako publish` (in-process, for payloads it stages), `tebako blksum` and `tfs blksum`. The last two exist because a runtime factory images with `tfs` and publishes through its own tooling — without a verb on the binary the leg already runs, a runtime pair can ship with no sidecar and lazy seeding can never engage (tebako#868). No second implementation of the group scheme exists, by construction.
- **The sidecar's own anchor** rides the existing channel, additively: the release shard/registry row gains `image.blksum: {filename, sha256}` (mirror-only, spec 03 §4's tier-3 rule); where the index is signed (spec 09 §5) the field is covered exactly like `image.sha256`. The sidecar is fetched and verified against that pin BEFORE the first range read; a sidecar whose bytes do not match its pin is the named `Sha256Mismatch` (70) — nothing mounts.
- **Trust at read (the protection point, exactly):** every Range GET's bytes are sha256-verified against the sidecar's group digest BEFORE they enter the block cache and BEFORE any byte is served. A group whose digest mismatches is dropped, refetched once, and on a second mismatch is the named `Sha256Mismatch` (70) — the mount fails closed, never a silent serve. This is today's verify-at-fetch law at group granularity; the per-run law holds unchanged (a cached group serves without re-verification — the cache is trusted once written, exactly like today's sealed entries). The user is therefore protected from the FIRST byte of the FIRST run — no execution of unverified bytes, ever.
- **Whole-file sha256 at seal (mandatory second anchor):** the seal pass (§5) assembles the entry's `.tfs` from the verified groups and verifies the whole-file sha256 against the original pin before the entry flips to SEALED. The `.sha256` sidecar written at seal is byte-identical in shape and meaning to today's anchor. Defense in depth, and what makes a sealed entry indistinguishable from an eagerly installed one.
- **Missing sidecar → LOUD eager fallback (locked).** Lazy is requested but the release publishes no sidecar (every pre-2026 image): the loader fetches the whole image exactly as today (whole-file sha256 verify — EQUAL trust strength, no downgrade), with a stderr warning + journal line (`event=lazy-fallback reason=blksum-missing`) naming the artifact. Never a hard error — every published image must stay runnable — and never silent (invariant 9; the spec 05 §4 stale-serve precedent for loud degradation).

© 2026 The Tebako Project

- **The cached-blksum arm (locked, with the §7 flip).** The loader fetches and pin-verifies the sidecar at INSTALL time regardless — the seed decision needs it — so the install also writes it into the entry (`<image>.blksum.json`, tmp+rename, BEFORE the descriptor's commit rename). The mount-open consults the cache FIRST: a verified hit is a zero-network open (nothing the exec phase owes the wire); a miss fetches as before and writes through (best-effort — the verified blksum is already in hand, so a cache-write failure never fails the mount); a cached file off its pin or torn is the named `Sha256Mismatch` (70) / `BlksumInvalid` (65), never a silent drop-and-refetch — the cache is a publisher-anchored document's copy. Two consequences the flip needs: the offline refusal for a FRESH seed fires at the install/pull phase (pre-exec) instead of surfacing as a mount-open 69 mid-exec, and the mount-open of every entry seeded after this arm performs no sidecar GET at all.

4. Cache design and the store-state amendment

Layout (inside the existing runtime entry, spec 05 §3 — LAZY_SEEDING state only):

FIGURE 2

```
runtimes/<lang>-<lv>-<ver>-<triplet>/
  tebako-runtime-<ver>-<lang>-<lv>-<triplet>[.exe]  # exe —
always whole, always verified (0755)
  sha256 / origin                                   # exe
markers, unchanged
  <image>.lazy.json                                 # the
seed descriptor — present ⇔ LAZY_SEEDING
  <image>.blksum.json                               # the
install-time sidecar cache (§3's cached-blksum arm)
  <image>.blocks/<NNNNNN>.blk                       #
verified group payloads (0444)
  <image>.tfs                                       #
appears ONLY at seal (0444)
  <image>.tfs.sha256 / <image>.tfs.origin           #
written at seal — today's shapes, unchanged
```

- **The seed descriptor** (`<image>.lazy.json`, versioned JSON, the blksum schema's sibling `schema/tpkg-lazy-seed-v1.schema.json`): source URL (the concrete origin), the whole-image sha256 pin, the blksum sidecar's sha256, `group_size`, `size_bytes`, group count. Written tmp+rename under the entry's flock (spec 05 §4) by the loader

© 2026 The Tebako Project

INSTEAD of the image download. A descriptor that fails validation at open is the named `LazyDescriptorInvalid` (65) — never a guess.

- **The block map IS the directory.** Present `*.blk` files are the seeded set; no separate bitmap, no journal. Open scans the directory and validates each group's size against the descriptor; a size-mismatched or unparseable group file is discarded and refetched.
- **Crash consistency:** per group, fetch → verify → write tmp → rename, under the entry's flock. A crash mid-write leaves a `.part` that the next open drops. A group file present = verified, by construction.
- **The store-law amendment (scoped, locked):** spec 05 §4's "a runtime is either fully installed or not" gains exactly one middle state. A LAZY_SEEDING entry (exe + markers present and verified, descriptor present and valid, image ABSENT) IS runnable — that is the feature — and counts as INSTALLED for resolution (the newest-compatible-cached pick, spec 05 §5, must see it, or every run re-resolves and re-seeds), but is reported distinctly by every listing surface (§7). The commit point for the lazy install is the descriptor's rename; the commit point for the SEALED entry stays the `.sha256` anchor's rename. No other store invariant moves: tmp+rename everywhere, per-entry flock (120 s, stale-lock hint), read-only artifacts, a run re-verifies nothing that is cached. `cache prune` removes a lazy entry like any other (descriptor, blocks, and all).
- **The exe is never lazy** and the runtime store entry's exe half installs exactly as today (the spec 05 §5 chain, the spec 06 §3 contract gate, the spec 09 §4 verification point — all before the seed descriptor is written).

5. The seal path and the state machine

States (named, on disk):

FIGURE 3

```
EMPTY → (loader: exe install + descriptor rename) → LAZY_
SEEDING
LAZY_SEEDING → (seal: all groups present → assemble → whole-
sha ok
               → .tfs + .sha256 rename → blocks +
descriptor + blksum
                 cache removed) → SEALED
SEALED ≡ today's ordinary entry — byte-identical layout,
anchors, semantics
```


© 2026 The Tebako Project

- **The background seal thread** lives in the DRIVER (inside the runtime exe — the process that holds the mount): spawned after the env image mounts, it fetches the missing groups in index order, one at a time, on the same byte source (and therefore the same verification). Its scheduling law: **on-demand reads always preempt** — a touching read's group fetch jumps the queue; the seal fills gaps around demand. It renders NO progress inside a running payload (stderr belongs to the payload after handoff — spec 11 §11 discipline; seal activity logs to tebako-log at debug, component `driver`).
- **Cancellation and resumption:** process exit simply stops the thread — the block cache persists; the next run (any package sharing the entry) resumes from the directory scan. Seal is best-effort forever: a run NEVER blocks on it, never fails because of it (a seal-side fetch failure logs and retries next run). `TEBAKO_LAZY_SEAL=0` disables the background thread (the debug/airgap shape; on-demand seeding still works).
- **The commit:** with all groups present, seal assembles `<image>.tfs` by concatenation into a tmp file, streams the whole-file sha256 over it (one pass, constant memory — the spec 05 §6 discipline), compares the original pin (mismatch → named 70, the assembled tmp dropped, the groups kept — a group verified against a publisher-anchored sidecar cannot silently disagree with the pin; a disagreement is evidence of a publisher-side or anchor-side fault and is reported, never healed), renames `.tfs` (0444), then `.tfs.sha256` + `.tfs.origin` (the anchor's rename is the commit point), then removes the blocks and the descriptor. A crash anywhere before the anchor rename leaves LAZY_SEEDING, fully resumable.
- **The explicit verb:** `tebako cache seal [<entry>|--all]` runs the same pass synchronously from the toolchain — full tebako-term ProgressSet rendering (§7). Operators who want first-run-lazy but warm-cache-complete run it at their convenience.
- Subsequent mounts of a SEALED entry take the ordinary whole-file path — the lazy machinery is out of the loop entirely.

6. Failure semantics (named, never a partial mount)

- **Mount-open failures** (prefix or record-table fetch fails, sidecar pin mismatch, descriptor invalid): the env image mount fails BEFORE the boot proceeds — the driver's existing unmount-everything law (spec 17 §1; `crates/tebako-driver/src/driver.rs`) applies verbatim: never a partial mount, the named error surfaces.
- **Network partition mid-mount:** a touching read whose group cannot be fetched after the retry law exhausts returns **EIO on that read** (the errno channel), named on the tebako-log with URL, group index, and the transport cause. The payload sees

© 2026 The Tebako Project

an ordinary IO error; the mount and every cached group remain valid; the next run resumes. Never a fabricated zero-fill, never a silent short read of unverified bytes.

- **Retry/backoff:** tebako-http's law, unchanged and unrepeated — Retry-After honored exactly, the 60 s × 2^n hintless exponential, THROTTLE_ROUNDS, DOWNLOAD_ATTEMPTS (spec 05 §6, `crates/tebako-http`). A mid-stream group failure retries THE GROUP from zero (a partial group would fail its digest anyway — the pipeline's artifact rule at group scale).
- **`TEBAKO_OFFLINE=1`:** cache-or-named-error at group granularity. Cached groups serve (a fully-seeded-touched working set runs offline); a miss is EIO on the touching read, or — at mount-open, when the prefix is not yet cached — the named offline refusal. Unchanged semantics, finer grain.
- Exit codes (reuse, never extend — the 65–79 space stands):

TABLE 1

condition	class
descriptor invalid, lazy config malformed, non-HTTPS source	65 usage
mount-open fetch failure; offline miss; server unusable	69 unavailable
sidecar pin mismatch; group digest mismatch (2nd); whole-sha mismatch at seal	70 integrity
cached blksum torn (65) / off its pin (70) — the §3 cache arm	65 / 70
store IO (lock, block write, assemble, rename)	74 IO

spec 09's 71/72 bind the sidecar's signed-index chain exactly as they bind the image's today; `TEBAKO_REQUIRE_SIGNED=1` fails closed before the first range read, unchanged.

- **HTTPS-only (locked):** the source URL is `https://` or `file://` (the test/airgap spelling), enforced by tebako-http's existing refusal — plus the spec 38 §8 LOOPBACK carve-out (`localhost`, `127.0.0.0/8`, `::1` over plain HTTP) for the §10 fixture only. There is no insecure-source spelling and never will be. TLS roots, proxy policy, and credentials ride tebako-http's one agent (spec 04 §4/§5) — no second client, and the credential book confines range requests exactly like whole-file GETs.

7. UX: mount mode, progress, doctor

- **The mount-mode chain (the flip LOCKED 2026-10-07, spec-00-style note).** The default is **lazy-on**: a runtime install on a release serving the lazy wire seeds the env image instead of fetching it whole. Resolution: `TEBAKO_RUNTIME_LAZY` (env) over

© 2026 The Tebako Project

`runtime_lazy` in `~/.tebako/config.yaml` over the product default ON — env wins per key (the `TEBAKO_FETCH_JOBS` / spec 04 §4 precedence rule), an unparseable value is a NAMED error (65), never a silent clamp. Eager stays first-class as the opt-out (`TEBAKO_RUNTIME_LAZY=0`, `runtime_lazy: false`). The division of authority is locked: the PUBLISHER controls enablement only — it ships the blksum sidecar or not (§3; there is no publisher forcing knob and never will be, because correctness is mode-independent); the USER/MACHINE controls the effective mode through the chain above. Hard rules outrank every tier, unchanged: `TEBAKO_OFFLINE=1` is cache-or-named-error (a fresh seed refuses at the pull phase, §3's cached-blksum arm); fat packages never fetch; a sidecar-less release takes the loud eager fallback. The flip's gate was §10's parity data; the landed numbers are recorded there.

- **The bundle-era gate (spec 36 §3's co-publish).** Both loaders (shim and bootstrap) evaluate the lazy arm BEFORE the spec 36 §4 era branch. On a shard declaring `bundle`, the arm engages ONLY when the shard also carries the co-publish witness (`per_file_assets: true`, runtime-manifest MINOR 2 — the exe/image/dll are then served standalone by construction); a bundle-declaring shard without the witness stays eager by construction (its one tar fetch IS the runtime). When the arm engages on a bundle-declaring shard, the install fetches the exe (+ the windows DLL) by the shard's per-file pins and seeds the image; the bundle is never fetched. Every other rule of this section is unchanged: a missing sidecar is the loud eager fallback, a torn pin the named error, a mismatch exit 70. The eager default on bundle-era lines stays the bundle fetch.
- **The host-tier gate (tebako#853).** The lazy wire is chosen only where the HOST's boot tier can read a seeded entry (`tpkg::lazy::host_boot_reads_lazy` — the single owner of that answer, consulted by the shim and both bootstrap planners). The POSIX tiers read through the lazy byte source and qualify; the windows materialize tier (spec 17 §7) extracts a host tree from the env image FILE, and a seeded entry has none until the seal commits — so on windows the opt-in takes the loud eager fallback (`event=lazy-fallback reason=windows-materialize-tier`, the journal line included) and the first run works; an OCI source names its own host-independent cause first (`reason=oci-bundle-era-only`). A seeded entry that arrives on a windows store by other means still refuses by name in the driver, unchanged.
- **Progress.** The loader's lazy first run renders through tebako-term (the spec 06 §5/§5a wiring rule — nothing else prints download progress): the exe's ordinary artifact bar, then one line — `runtime env image <name> (<size>) — lazy: seeding on demand, sealing in background`. `tebako cache seal` renders the block map as a ProgressSet slot — `sealing <image> [<bar>] 62% (groups 831/1340, 3.1 MB/s)` — group-accurate, not estimated. Plain mode and `TEBAKO_NO_PROGRESS=1` behave per spec 06 §5a, unchanged.
- **`tebako doctor`** (spec 35) store section gains the seed state per runtime entry: `LAZY_SEEDING — 231/1340 groups (17%), source <host>, seal resumable` or `SEALED`; a descriptor/block-dir inconsistency is reported by name (never healed silently).
- **`tebako cache list`** marks lazy entries (`(seeding 17%)`); the origin marker carries the source URL as today.

© 2026 The Tebako Project

8. Interaction with spec 38 (OCI)

The byte source is transport-shaped, not URL-shaped: it consumes a range-fetch closure. An OCI blob GET (`/v2/<repo>/blobs/<digest>`, spec 38 §5) supports Range on every conformant registry, and the blob digest IS the artifact sha256 (spec 38 §3's trust-anchor equivalence) — so a range source over an OCI blob needs NO grammar, NO divergence: the OCI adapter supplies its closure (Bearer flow and confinement of spec 38 §6 included), the blksum sidecar publishes as one more §3 artifact class.

The closure arm (locked 2026-10-01 with PR 6). The OCI client's blob fetch carries the range arm: one positioned GET (`Range: bytes=<offset>-<end>`, the caller-decided credential header attached verbatim per dance attempt), answering 206 with a Content-Range validated against the request (exact start, exact length) or 200 — the §3 loud-eager-fallback signal, byte-identical in meaning with the plain-HTTPS transport's. A range window carries no whole-blob digest check by construction: the per-group digests of §3 are the integrity layer, exactly as over plain HTTPS. The blksum sidecar's OCI spellings are spec 38 §3's (that section owns the class table and the tag rules): artifactType `application/vnd.tebako.blksum.v1`, layer media type `application/vnd.tebako.blksum.v1+json`, the sibling digest-tag `sha256-<64 hex of the IMAGE blob's digest>.blksum.json` with the `org.tebako.blksum.subject` annotation naming the image digest. The sidecar's layer digest IS the blksum document's sha256 — the seed descriptor's `blksum_sha256` pin binds it with no second file, and a resolve of the sibling tag that finds nothing is the §3 blksum-missing signal (the loud eager fallback), never an error.

Recorded, not committed (the run-time wiring). PR 6 ships the transport arm, the sidecar class, the resolve-side sidecar resolution, and the publish push — the pieces any OCI-staged lazy entry needs. The two downstream legs land with their own specs' PRs: OCI runtime distribution itself is spec 38's PR 4 (runtime bundles over OCI), and the driver's mount-open OCI arm (state-detecting an OCI-sourced seed descriptor, the run-time credential chain for a mount that outlives the install) is designed with that leg — this spec commits the closure shape only.

9. What does NOT change (locked)

- **Published image bytes.** No format change, no new in-image section, no format_id, no trailer field. Every LimniFS image ever published is lazily mountable as-is (§1).
- **The `.sha256` anchor.** Sealed entries are byte-identical in layout, anchors, and semantics with eagerly installed ones (§5).
- **The mount-table law.** Longest-prefix dispatch, duplicate-point EEXIST, unmount-everything on failure (spec 11 §2, spec 17 §1) — lazy changes where bytes come from, never how mounts compose.

© 2026 The Tebako Project

- **The transport window.** A demand fetch runs with the engine's read lock held, and the fetch's OWN machinery may call back into the interposed libc surface — on macOS the resolver reads `networkd_settings` through open, which would re-enter the engine behind the lock: a deadlock (tebako#851: one thread in the VFS read, the other in `getaddrinfo → … → open_impl`). While a fetch is in flight the shim therefore PASSES EVERY interposed call THROUGH to the host; the window is PROCESS-WIDE (the resolver runs on its own thread), not thread-local. The window is the transport's own machinery, never a payload's read; a concurrent payload read on another thread during a fetch is the known narrow exposure, and the follow-up is a lock-free mount-prefix fast path that scopes the bypass to paths outside every mount.
- **The mount-table identity.** A lazy env-image mount serializes into `TEBAKO_TFS_MOUNTS` by its entry path, exactly like a file mount. A re-establishing consumer — the interpreter's preload shim, and every exec'd child — state-detects that path the same way the driver does: the sealed file mounts whole, the ABSENT file with a valid descriptor beside it mounts the caching range source, and neither file nor descriptor keeps the ordinary named open error. The lazy promise holds at both layers: only the groups demand touches are fetched.
- **The handoff grammar.** `TEBAKO_RUNTIME_IMAGE=<path>` and the `--tebako-image` triples are byte-identical (spec 17 §1); the path names the entry whose on-disk state (§4) the driver reads. No contract-version bump: `TEBAKO_RUNTIME_LAZY` is an operator env in the `TEBAKO_MOUNT_ROOT` / `TEBAKO_JAIL` class (spec 17), not a handoff-semantics change (spec 06 §6's bump rule). A pre-lazy driver handed a LAZY_SEEDING entry fails CLOSED — the image file is absent, the existing named unavailable error (69) fires, and the remedy is an eager run or a runtime rebuild; never a mis-mount.
- **The 3 MiB bootstrap gate.** The mount happens in the DRIVER inside the runtime exe (spec 17; the bootstrap never mounts and never links TFS — spec 20 §1). The lazy machinery (byte source, block cache, seal thread) lives in `tfs` + `tebako-driver` + `tebako-http` — compiled into the runtime exe and the toolchain, neither size-gated. The bootstrap's only new code is the seed-record arm of its existing resolve-install path (write the descriptor instead of streaming the image — a tebako-resolve plan item with a lazy commit closure); the CI size gate measures and enforces the delta, as always (invariant 2). The same machinery is also linked into the preload shim (the interpreter's own engine instance re-establishes the mount, §9's identity rule), which is not size-gated either.

© 2026 The Tebako Project

10. Test plan (the tiers of spec 14; each failure above maps to one tier)

- Unit (tfs / tpkg / tebako-http): range math and group indexing (edge: short final group); descriptor and blksum schema round-trips, every malformed class by name; Content-Range parsing, 206-vs-200 detection, ETag/If-Range revalidation; the directory-scan block map (size-mismatch discard, tmp drop); the state machine's transitions and commit points; config/env precedence.
- Property (proptest, the existing harnesses): arbitrary read patterns over a mock byte source answer IDENTICALLY to whole-image reads of the same bytes (the parity property); arbitrary crash points (kill between every fsync/rename) leave a resumable LAZY_SEEDING or a complete SEALED, never a third state; arbitrary garbage descriptors never panic and never parse silently.
- Contract (in-process HTTP fixture — CI tooling, never shipped; the spec 38 §8 loopback carve-out): a Range-capable fixture server with artificial latency, injected 500s/429s, mid-stream kills, truncated bodies, 200-instead-of-206, and byte-level corruption per group. Legs: cold mount → on-demand seeding → answers equal the file-mounted golden tree (spec 20 §8's backend-pair parity class, the dwarfs/limnifs oracle extended to lazy-vs-file); group digest mismatch → refetch → named 70; partition mid-boot → named 69; partition mid-run → EIO on the touching read only; offline with partial cache; seal from partial, seal resumption after kill, seal commit ordering; the 200-fallback (whole body arrives — loud eager path).
- E2e (`ci/`, tiered): shim dispatch with lazy on — first cold run boots the entrypoint against a fixture-hosted runtime, second run serves touched paths with the fixture DOWN (touched-set offline proof), `tebako cache seal` completes and the third run is ordinary; the standalone bootstrap leg (size gate asserted); the touch-everything pathology (a full-tree walk: total fetched bytes ≤ eager bytes + one sidecar + per-group request overhead, bounded and recorded — never worse than eager × (1 + ε)); the blksum-missing image takes the loud eager fallback; `TEBAKO_REQUIRE_SIGNED=1` pass/fail legs per spec 09.
- Parity data for the default flip (spec 27) — LANDED. The tebako-bench harness's lazy arm ran the full 7-leg platform matrix green (bench run 37517037555, zero failed runs): warm second-run parity 1.00× everywhere; cold one-time streaming overhead bounded at +0.56 s (windows) to +5.5 s (linux-musl-x86_64) on the loopback fixture — the ε of the pathology law (total fetched bytes ≤ eager bytes + one sidecar + per-group request overhead), and the number a real network inverts, since first run fetches only the touched groups; lazy cold RSS below eager's (e.g. 25 vs 34 MiB at boot). No regression class in the warm or pathology arms. On this data the §7 flip shipped (locked 2026-10-07): the default is lazy-on, the eager opt-out is `TEBAKO_RUNTIME_LAZY=0` / `runtime_lazy: false`.

© 2026 The Tebako Project

11. Open questions (each with this spec's recommendation)

1. **Backend vs byte source.** Recommended and locked (§2): a mount-source kind below the format backend. The issue's "http-range backend" is the right instinct at the wrong layer — transport is not a format.
2. **Whole-file sha at seal vs per-block digests at read.** Recommended and locked (§3): per-group digests at read (publisher-authored, channel-anchored sidecar) + mandatory whole-file verify at seal. Seal-only verification executes unverified bytes — rejected on the threat model.
3. **Group size.** Recommended and locked: 4 MiB of image bytes. Smaller multiplies request count against rate-limited hosts; larger wastes fetch on sparse touches. One constant, tunable only by a measured follow-up (the §10 bench arms).
4. **Missing sidecar.** Recommended and locked (§3): loud eager fallback (equal trust, warned, journaled) — never a hard error against the long tail of already-published images.
5. **Lazy payload slices.** Recommended: env image v1; payloads ride the identical machinery behind the same flag later (their registry rows already carry sha256 anchors; the sidecar field is additive). Recorded, not committed.
6. **Dwarfs over range.** Recommended: recorded hook only (§2) — the C++ callback boundary is real work and limnifs is the default; decided by measured demand, not speculation.
7. **Seal assembly.** Recommended and locked (§5): assemble from the verified groups (no second network pass) — the whole-file sha is computed over the local concatenation.
8. **The exe.** Recommended and locked: always eager (kernel exec needs a whole file). The exe is the minority of the pair's bytes; the env image is the win.
9. **A second verification algorithm (BLAKE3 groups).** Recommended: no. sha256 everywhere keeps ONE anchor vocabulary with the store, the sidecars, and spec 38's layer digests; LimniFS's BLAKE3 ids remain image-internal integrity (spec 20 §7's recorded addendum, unchanged).

Implementation plan (spec PR first; each PR independently mergeable)

1. **PR 1 — the spec.** This document + the `00-INDEX.md` entry + `schema/tpkg-blksum-v1.schema.json` + `schema/tpkg-lazy-seed-v1.schema.json`. No code.

© 2026 The Tebako Project

2. **PR 2 — the transport.** `crates/tebako-http/src/lib.rs`: the Range GET entry point (Range header, 206/Content-Range validation, If-Range/ETag revalidation, the loopback carve-out), unit + contract tiers with the fixture server.
3. **PR 3 — the byte source and the lazy store record.** `crates/tpkg/src/lazy.rs` (descriptor + blksum models, the store-record read/write/scan under `tpkg::runtime_store`), `crates/tfs/src/source.rs` + `crates/tfs/src/source_remote.rs` (the byte-source seam, the caching remote source, `backend-remote` feature), `crates/tfs/src/mount.rs` (the fifth mount-source kind), `crates/tfs/src/backends_limnifs.rs` (the sourced open — prefix + record-table fetch), the limnifs-core paged-`SlabSource` pin (upstream first). Unit/property tiers + the parity class.
4. **PR 4 — the loader + driver wiring.** `crates/tebako-resolve/src/plan.rs` (the lazy commit closure writing the seed record), the shim/bootstrap resolve paths (`TEBAKO_RUNTIME_LAZY` + config), `crates/tebako-driver/src/driver.rs` (state-detecting env-image mount, the seal thread, EIO mapping), `crates/tebako-cli/src/doctor.rs`

   — the cache list/seal surfaces. E2e legs of §10.
5. **PR 5 — the publish path.** `crates/tebako-cli/src/publish.rs` (blksum generation in-process), the shard/registry additive `image.blksum` field (validation + schema), the factory release legs (factory repos, stacked on PR 5's product half).
6. **PR 6 — OCI range + the flip data.** The spec 38 §5 closure arm; the tebako-bench lazy arm (spec 27 harness); the default-flip PR only after §10's rule is met.

© 2026 The Tebako Project